La mise en page avec CSS

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Les bonnes pratiques CSS **3**

A. Les bonnes pratiques CSS 3

B. Exercice : Quiz 6

III. Flexbox **7**

A. Flexbox 7

B. Exercice : Quiz 12

IV. L'essentiel **12**

V. Auto-évaluation **13**

A. Exercice 13

B. Test 14

Solutions des exercices **15**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée : 1 heure.

Prérequis : Notions HTML, CSS.

Environnement de travail : PC ou Mac avec 8 Go de ram minimum.

Contexte

Le CSS est un langage de style utilisé pour définir la présentation d'un site web. Il permet de contrôler l'apparence de chaque élément HTML et de créer des designs esthétiquement agréables et cohérents. Cependant, l'utilisation du CSS peut parfois devenir complexe, surtout lorsque le site web comporte plusieurs pages et éléments à styliser. Il est donc important de suivre certaines bonnes pratiques pour faciliter la gestion du CSS et améliorer la qualité du code. Nous aborderons donc, dans ce cours, certaines de ces bonnes pratiques pour aider les développeurs à créer des styles CSS efficaces et maintenables.

De plus, le design d'un site web est primordial pour attirer et retenir l'attention des visiteurs. Pour autant, la création d'une mise en page complexe et « *responsive* » peut parfois s'avérer difficile, surtout lorsqu'on doit s'adapter à différentes tailles d'écran. Heureusement, Flexbox est là pour nous simplifier la vie ! Cette technique de mise en page CSS permet de créer facilement des mises en page flexibles et adaptatives pour les éléments HTML. En comprenant et en utilisant les différentes propriétés du conteneur flex et des éléments flex, il est possible de créer des designs esthétiquement agréables et harmonieux, tout en simplifiant le processus de mise en page dite responsive.

II. Les bonnes pratiques CSS

A. Les bonnes pratiques CSS

Organisation et structure des feuilles de style

Afin de maintenir un code CSS efficace et facile à comprendre, une bonne organisation est nécessaire pour éviter les erreurs et faciliter la navigation lors de la maintenance du code. Parmi les bonnes pratiques pour organiser et structurer votre code CSS, on peut citer la bonne utilisation des commentaires, la séparation des règles similaires en sections thématiques et l'adoption d'une convention de nommage cohérente pour les classes et les IDs, tout en veillant à l'accessibilité et à quelques autres points que nous aborderons ci-dessous.

Méthode **Les commentaires**

Utiliser des commentaires pour décrire les sections du code et faciliter la compréhension est une excellente pratique à adopter. Les commentaires sont des lignes de code qui ne sont pas interprétées par les navigateurs web, mais qui peuvent être utilisées pour **expliquer le code**. Dans les feuilles de style CSS, ils aident à décrire les sections du code, à clarifier les intentions du développeur et à faciliter la compréhension pour les autres membres de l'équipe ou pour vous-même lors de la maintenance du code. Ils peuvent également aider à expliquer pourquoi certaines décisions ont été prises. Pour rédiger des commentaires efficaces, soyez concis et utilisez des mots-clés pour faciliter la recherche dans le code.

Exemple

Les commentaires CSS commencent par `/*` et se terminent par `*/`. Voici un exemple :

```
/* Barre latérale */
.aside-container {
width: 30%;
}

```

```
aside {
border: 1px solid green;
background-color: lightgreen;
}

/* Espacements et bordures arrondies */
section, aside, article {
padding: 20px;
margin-bottom: 20px;
border-radius: 10px;
}
```

Les commentaires sont souvent écrits en anglais ; même si en France, ce n'est pas systématique, c'est une bonne pratique. Par souci de clarté, nous les utiliserons en français dans ce cours.

Méthode **Regrouper les règles similaires et les séparer par des sections thématiques**

Il est recommandé de regrouper les règles similaires et de les séparer par des sections thématiques pour maintenir une organisation claire et lisible du code. Cela peut aider à identifier les erreurs plus rapidement et à éviter les conflits de style, tout en facilitant la navigation et la maintenance du code.

Exemple

Vous pouvez regrouper les styles pour les en-têtes de page séparément des styles pour les paragraphes, ou les règles concernant la typographie, les couleurs, la mise en page, etc. Cela améliore la lisibilité, et une organisation structurée permet une maintenance plus facile du code.

```
/* Typography */
body {
font-family: Arial, sans-serif;
font-size: 16px;
line-height: 1.5;
}

h1, h2, h3, h4, h5, h6 {
font-weight: bold;
}

/* Colors */
body {
background-color: #ffffff;
}

a {
color: #f00;
text-decoration: none;
}

/* Layout */
header {
/* ... */
}

nav {
/* ... */
}

```


Tous droits réservés à STUDI - Reproduction interdite

```
    /* ... */
```

Méthode **Adopter une convention de nommage cohérente pour les classes et les IDs**

Choisir une convention de nommage cohérente pour les classes et les IDs est important pour maintenir une structure claire et lisible du code. Il existe plusieurs méthodologies de nommage, telles que **BEM** (*Block, Element, Modifier*), **OOCSS** (*Object-Oriented CSS*) et **SMACSS** (*Scalable and Modular Architecture for CSS*), qui peuvent aider à faciliter la compréhension et la maintenance du code. Utiliser une convention de nommage cohérente peut aider à éviter les erreurs et à rendre le code plus facile à comprendre pour les développeurs qui travaillent sur le même projet. Par exemple, en utilisant la méthodologie BEM, les noms de classe sont constitués d'un bloc, d'un élément et d'un modificateur, séparés par des doubles tirets (--) et des doubles underscores (__). Il est également conseillé d'utiliser des noms de classes et d'IDs qui reflètent leur fonction et leur utilisation sur la page web, tels que « *header* », « *footer* », « *navigation* », etc.

Exemple

```html
<!-- HTML -->
   <div class="card">
   <h2 class="card__title">Titre</h2>
   <p class="card__description">Description</p>
   <button class="card__button card__button--primary">Bouton</button>
   </div>
```

```css
/* Réinitialisation du navigateur */
* {
 margin: 0;
 padding: 0;
 box-sizing: border-box;
}

/* Styles généraux */
body {
 font-family: Arial, sans-serif;
}

/* En-tête */
header {
 background-color: #333;
 color: white;
 display: flex;
 justify-content: space-between;
 align-items: center;
 padding: 20px;
}

/* Logo et titre */
.logo {
 font-size: 24px;
 font-weight: bold;
 display: flex;
 width: 500px;
 justify-content: space-between;
}
```

Notez que l'utilisation de préprocesseurs CSS tels que Sass ou Less peut améliorer l'organisation et la gestion des feuilles de style en permettant la modularisation, l'importation de fichiers et d'autres fonctionnalités utiles.

Tous droits réservés à STUDI - Reproduction interdite

Sélecteurs et spécificités

Les sélecteurs d'IDs peuvent être utiles dans certaines situations, mais il est généralement préférable de les utiliser avec parcimonie. Les IDs sont conçus pour identifier un élément unique sur une page web, de sorte qu'ils ne peuvent être utilisés qu'une seule fois par page. Les IDs ont également une spécificité plus élevée que les classes, ce qui signifie qu'ils peuvent prendre le dessus sur les styles définis par les classes lorsqu'il y a un conflit.

Cependant, il peut y avoir des cas où vous voulez styliser un élément unique sur une page, et dans ce cas, vous pouvez utiliser un sélecteur d'ID. Par exemple, vous pouvez utiliser un ID pour styliser un élément qui ne se répète jamais sur la page, comme un en-tête unique ou un formulaire de contact.

En général, il est préférable de privilégier les **classes** pour styliser les **éléments** et de réserver les **IDs** pour les cas où vous avez besoin de styliser un **élément unique**.

Minimiser la spécificité et la complexité des sélecteurs est important, car plus un sélecteur est spécifique et complexe, plus il est difficile à maintenir et plus il est susceptible de causer des conflits de style. Il est préférable d'utiliser des sélecteurs simples et peu spécifiques pour styliser les éléments.

De plus, il est important d'éviter les sélecteurs universels et les combinaisons inutiles, car cela peut entraîner une surcharge inutile du navigateur, ce qui peut ralentir la performance de la page web. Il est préférable de cibler les éléments de manière précise en utilisant des sélecteurs pertinents pour chaque élément.

L'accessibilité en CSS

L'accessibilité en CSS est une considération importante pour assurer une expérience utilisateur optimale pour tous les utilisateurs. Un élément important est d'utiliser des **couleurs contrastées** pour améliorer la lisibilité du texte pour les personnes ayant des difficultés de vision ou de lecture de certaines couleurs. Les couleurs contrastées peuvent également améliorer l'expérience utilisateur pour tous les utilisateurs. Il est également important de prévoir des **styles pour les états de focus et de survol** des éléments interactifs, tels que les liens et les boutons.

Cela est particulièrement important pour les utilisateurs qui naviguent à l'aide d'un clavier ou d'un appareil d'assistance. Les styles pour les états de focus et de survol peuvent aider à indiquer visuellement à l'utilisateur où se trouve le focus sur la page et où se trouvent les éléments interactifs. La prise en compte des **préférences** des utilisateurs est également importante. Il est essentiel de réduire les animations pour les personnes sensibles aux mouvements afin de garantir une expérience utilisateur accessible pour tous les utilisateurs. Certaines personnes peuvent avoir des difficultés à suivre les mouvements rapides sur un écran, et la réduction des animations peut aider à améliorer leur expérience utilisateur.

Pour garantir une bonne **lisibilité** du texte, il est également essentiel de choisir une taille de police appropriée et une longueur de ligne optimale. Une taille de police trop petite ou des lignes de texte trop longues peuvent rendre la lecture difficile pour les utilisateurs, notamment pour ceux qui ont des problèmes de vision. La taille de police recommandée est généralement d'au moins 16 pixels, et une longueur de ligne optimale se situe entre 45 et 75 caractères par ligne. Il est important d'adapter la taille de la police et la longueur des lignes en fonction des besoins de votre audience et du contexte de votre contenu pour garantir une expérience utilisateur optimale.

B. Exercice : Quiz

[solution n°1 p.17]

Question 1

Pourquoi est-il important d'utiliser des commentaires dans votre code CSS ?

- ○ Pour que le navigateur puisse interpréter les styles correctement
- ○ Pour faciliter la compréhension du code et aider à sa maintenance
- ○ Pour augmenter la vitesse de chargement de la page

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Quelle méthodologie de nommage utilise des blocs, des éléments et des modificateurs, séparés par des doubles tirets (--) et des doubles underscores (__) ?

- OOCSS
- SMACSS
- BEM

Question 3

Quelle est l'importance de regrouper les règles similaires et de les séparer par des sections thématiques en CSS ?

- Pour réduire la taille du fichier CSS
- Pour faciliter la navigation et la maintenance du code
- Pour améliorer les performances du navigateur

Question 4

Quelle est la spécificité des sélecteurs d'ID par rapport aux classes en CSS ?

- Plus faible
- Égale
- Plus élevée

Question 5

Pourquoi est-il important d'utiliser des couleurs contrastées en CSS ?

- Pour améliorer l'esthétique du site web
- Pour faciliter la navigation sur le site web
- Pour assurer une bonne lisibilité du texte pour tous les utilisateurs

III. Flexbox

A. Flexbox

Découverte du Flexbox

Le Flexbox est une technique de mise en page CSS qui permet de créer des mises en page flexibles et adaptatives pour les éléments HTML. Elle a été introduite en CSS3 pour répondre aux difficultés de mise en page rencontrées avec les méthodes de mise en page précédentes, telles que les flottants et les positionnements CSS. Le Flexbox permet de contrôler efficacement la disposition des éléments dans un conteneur en utilisant les propriétés CSS pour **aligner, distribuer et réorganiser** les éléments selon différents critères.

Les éléments Flexbox s'adaptent automatiquement à la taille de l'écran ou à la taille de leur conteneur, ce qui les rend particulièrement adaptatifs et flexibles. Les éléments peuvent être facilement alignés sur l'axe principal et l'axe secondaire à l'aide des propriétés CSS justify-content et align-items et peuvent être facilement réorganisés en utilisant la propriété CSS order.

Le Flexbox est une méthode de mise en page CSS qui diffère des autres méthodes. Comparé aux flottants, le Flexbox permet une meilleure gestion de l'alignement vertical des éléments et peut être utilisé pour créer des mises en page plus complexes. Il est plus flexible et plus facile à utiliser que le positionnement CSS pour créer des mises en page adaptatives. Mais surtout, il est plus simple et plus léger en matière de code que le Grid, même s'il peut être moins efficace pour la création de mises en page complexes et structurées.

Tous droits réservés à STUDI - Reproduction interdite

Méthode **Le conteneur flex**

Le conteneur flex est l'élément parent qui englobe tous les éléments flex (flex items). Pour créer un conteneur flex, vous devez définir la propriété CSS « *display* » de l'élément parent sur « *flex* » ou « *inline-flex* ». Cela permet d'appliquer des propriétés spécifiques à Flexbox sur cet élément et ses enfants.

Les éléments flex sont les enfants directs d'un conteneur flex. Ces éléments sont automatiquement organisés selon les propriétés définies sur le conteneur flex. Les éléments flex peuvent être manipulés individuellement à l'aide de propriétés spécifiques aux éléments flex.

Flexbox repose sur 2 axes : l'axe principal (*main axis*) et l'axe transversal (*cross axis*). L'axe principal est l'axe le long duquel les éléments flex sont disposés dans le conteneur. L'axe transversal est perpendiculaire à l'axe principal.

- L'axe principal est déterminé par la propriété « *flex-direction* » du conteneur flex. Par défaut, sa valeur est « *row* », ce qui signifie que les éléments flex sont disposés horizontalement, de gauche à droite. Si la valeur est « *column* », les éléments seront disposés verticalement, de haut en bas.
- L'axe transversal est perpendiculaire à l'axe principal. Si l'axe principal est horizontal («*row*»), l'axe transversal sera vertical, et inversement.

Comprendre ces axes est crucial pour manipuler correctement les éléments flex à l'intérieur du conteneur et pour ajuster leur alignement et leur espacement en fonction des besoins de mise en page.

Exemple

```
    <div class="parent">
       <div class="item">Item 1</div>
       <div class="item">Item 2</div>
       <div class="item">Item 3</div>
   </div>
```

```
 .parent {
    display: flex;
    color: blue;
 }
```

En exécutant ce code, vous aurez Item 1, Item 2 et Item 3 sur la même ligne, comme ceci :

Item 1 Item 2 Item 3

Méthode **Propriétés du conteneur flex**

La propriété « *flex-direction* » est une propriété clé pour la mise en page Flexbox, elle définit l'axe principal et la direction dans laquelle les éléments flex sont disposés. Il existe 4 valeurs possibles pour cette propriété: « *row* », « *row-reverse* », « *column* » et « *column-reverse* ». La valeur par défaut est « *row* », qui dispose les éléments horizontalement de gauche à droite.

La valeur « *row-reverse* » dispose les éléments horizontalement de droite à gauche. La valeur « *column* » dispose les éléments verticalement de haut en bas. Enfin, la valeur « *column-reverse* » dispose les éléments verticalement de bas en haut. En utilisant cette propriété, les développeurs peuvent contrôler facilement la direction dans laquelle les éléments flex sont disposés.

Exemple

```
.parent {
      display: flex;
      flex-direction: column-reverse;
      color: blue;
  }
```


Tous droits réservés à STUDI - Reproduction interdite

En appliquant ce code au HTML du bloc méthode : conteneur flex, vous deviez avoir de haut en bas Item 3, Item 2 et Item 1 comme ceci :

Item 3

Item 2

Item 1

Méthode **Flex-wrap**

La propriété « *flex-wrap* » est une autre propriété importante de la mise en page Flexbox. Elle contrôle si les éléments flex doivent être enveloppés (*wrapped*) sur plusieurs lignes ou colonnes en fonction de l'espace disponible. Cette propriété a 3 valeurs possibles : « *nowrap* », « *wrap* » et « *wrap-reverse* ». La valeur par défaut est « *nowrap* », qui signifie que les éléments flex sont disposés sur une seule ligne ou colonne. La valeur « *wrap* » enveloppe les éléments flex sur plusieurs lignes ou colonnes si nécessaire. La valeur « *wrap-reverse* » enveloppe également les éléments flex sur plusieurs lignes ou colonnes, mais dans l'ordre inverse.

Exemple

Remplaçons Item 1 par un texte un peu plus long :

Lorem, ipsum dolor sit amet consectetur adipisicing elit. Maxime molestiae officiis cupiditate natus. Earum distinctio quaerat necessitatibus eos tenetur ullam vero, quo rem minus eligendi, quisquam voluptatem voluptatibus veniam animi?	**Item 2**	**Item 3**

Ajoutons la propriété « *wrap* » :

Lorem, ipsum dolor sit amet consectetur adipisicing elit. Maxime molestiae officiis cupiditate natus. Earum distinctio quaerat necessitatibus eos tenetur ullam vero, quo rem minus eligendi, quisquam voluptatem voluptatibus veniam animi?	
Item 2	**Item 3**

Dans cet exemple ci-dessus, nous comprenons comment les éléments flex sont enveloppés sur une seule ligne. Voici le code qui permet de passer de la valeur « *nowrap* » par défaut à « *wrap* » :

```
.parent {
display: flex;
flex-wrap: wrap;
}
```

Notez que la propriété « *flex-flow* » est une propriété raccourcie pour définir à la fois « *flex-direction* » et « *flex-wrap* ». Voici un exemple d'utilisation :

```
.parent {
display: flex;
flex-flow: column wrap; /* Les éléments flex seront disposés en colonnes et enveloppés si nécessaire */
}
```

Méthode **Justify-content**

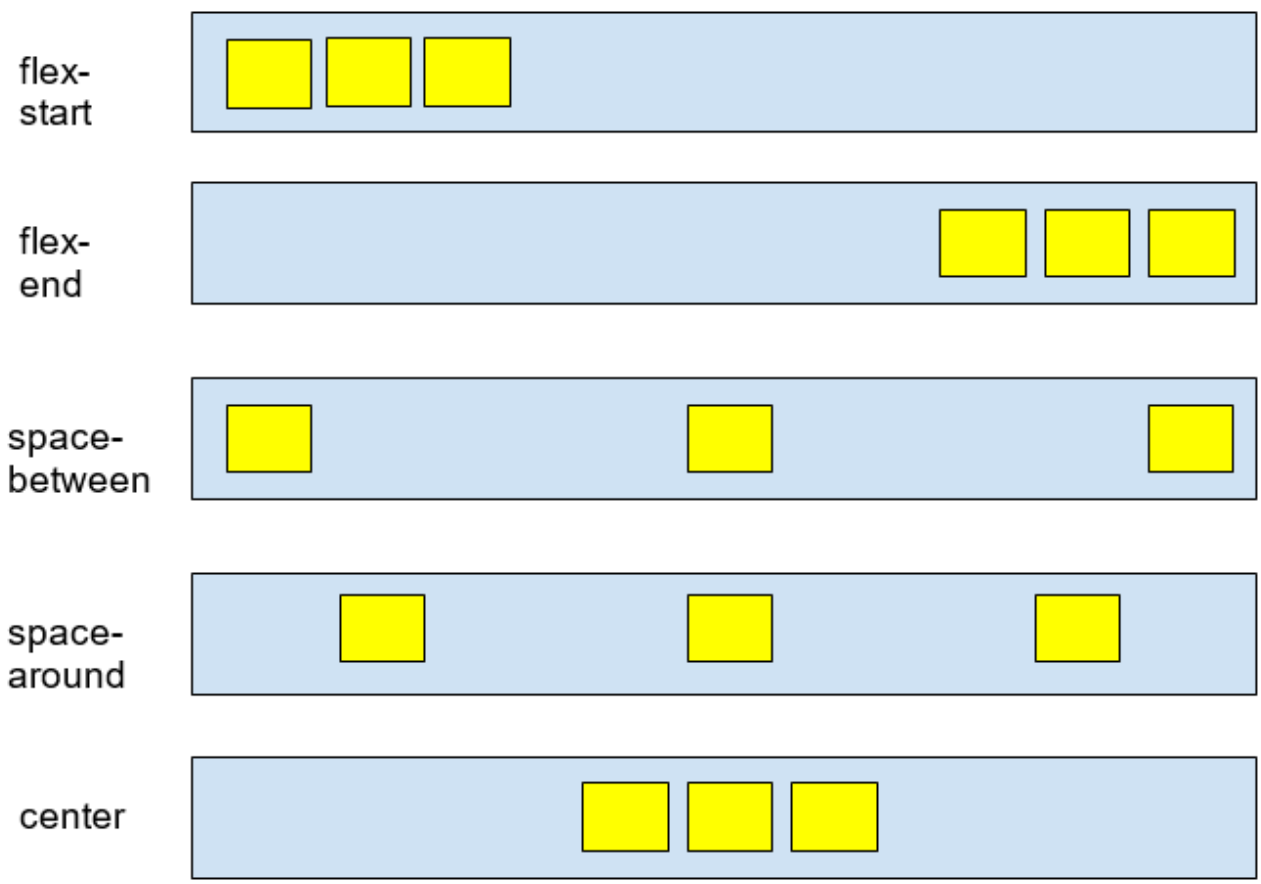
La propriété CSS « *justify-content* » est utilisée pour aligner horizontalement les éléments flex le long de l'axe principal du conteneur Flexbox. Les valeurs possibles sont flex-start, flex-end, center, space-between, et space-around. « *flex-start*» aligne les éléments sur le bord du début de l'axe principal, tandis que « *flex-end* » les aligne

Tous droits réservés à STUDI - Reproduction interdite

sur le bord de la fin de l'axe principal. « *center*» les centre horizontalement sur l'axe principal. « *space-between* » répartit uniformément les éléments avec un espace égal entre eux, tandis que « *space-around* » répartit uniformément les éléments avec un espace égal autour d'eux.

La propriété « *justify-content* » est particulièrement utile pour aligner les éléments flex de manière uniforme et professionnelle. Les développeurs peuvent l'utiliser pour aligner les éléments flex dans une disposition row ou column, ce qui permet de créer une mise en page esthétiquement agréable et harmonieuse.

Exemple



Dans le schéma ci-dessus, vous pouvez constater comment justify-content se comporte avec les valeurs les plus courantes en « *display: flex* », c'est-à-dire en disposition row par défaut.

Méthode **Align-items**

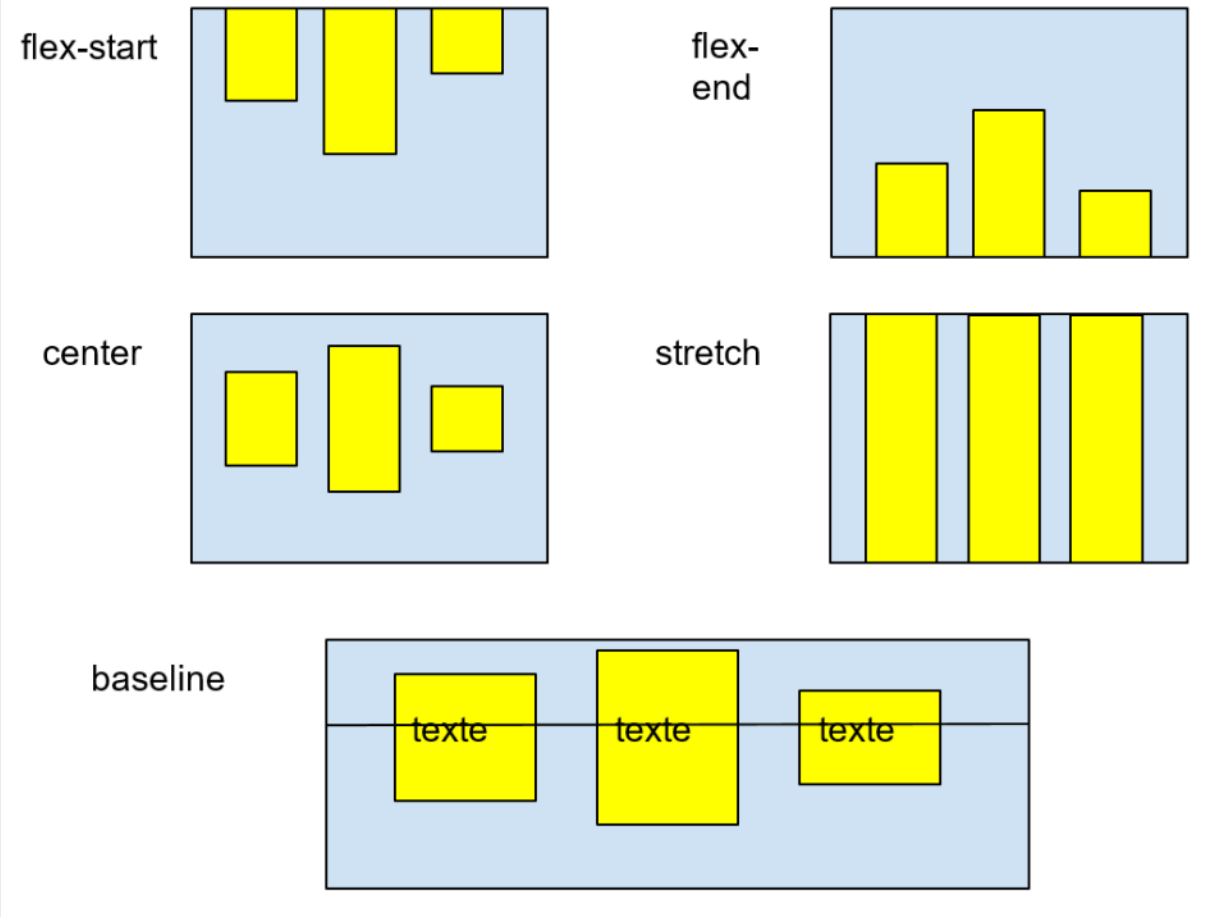
La propriété CSS « *align-items* » est utilisée pour aligner verticalement les éléments flex le long de l'axe transversal du conteneur Flexbox, c'est-à-dire perpendiculairement à l'axe principal. Cette propriété est l'équivalent de justify-content, mais pour l'alignement vertical des éléments flex.

En combinant align-items avec justify-content, les développeurs peuvent aligner à la fois horizontalement et verticalement les éléments flex pour créer des mises en page esthétiquement agréables et harmonieuses. Par exemple, si un conteneur Flexbox est configuré pour disposer les éléments horizontalement (« *flex-direction: row* »), la propriété align-items contrôlera l'alignement des éléments le long de l'axe vertical.

Tous droits réservés à STUDI - Reproduction interdite

Les valeurs possibles pour align-items comprennent flex-start, flex-end, center, baseline et stretch. « *flex-start* » aligne les éléments sur le bord du début de l'axe transversal, tandis que « *flex-end* » les aligne sur le bord de la fin de l'axe transversal. « *center* » les centre verticalement sur l'axe transversal, tandis que « *baseline* » les aligne sur leur ligne de base commune. « *stretch* » étire les éléments flex pour remplir l'espace disponible sur l'axe transversal.

Exemple



Dans le schéma ci-dessus, différentes valeurs d'align-items ont été attribuées en complément à display: flex.

En conclusion, Flexbox est un système de mise en page puissant et flexible qui simplifie grandement le processus de création de mises en page responsives et adaptatives. En comprenant et en appliquant les différentes propriétés du conteneur flex et des éléments flex, il est possible de créer des mises en page complexes et d'ajuster facilement l'alignement, l'ordre et l'espacement des éléments à l'intérieur d'un conteneur.

Parmi les nombreux avantages de Flexbox, on trouve sa capacité à s'adapter aux différentes tailles d'écran et de résolutions, ce qui facilite la création de designs réactifs. Par exemple, en utilisant la propriété « *flex-wrap* », les éléments peuvent être réorganisés automatiquement sur plusieurs lignes ou colonnes lorsque l'espace est limité. De même, avec « *justify-content* » et « *align-items* », il est simple d'aligner les éléments selon différents critères, sans avoir à recourir à des techniques de positionnement compliquées ou à des hacks CSS.

Un autre avantage de Flexbox est sa compatibilité avec les navigateurs modernes, ce qui en fait une solution fiable et largement utilisée pour la mise en page sur le Web. Les propriétés de Flexbox sont également faciles à comprendre et à appliquer, permettant aux développeurs de créer rapidement des mises en page élégantes et fonctionnelles.

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice : Quiz

[solution n°2 p.18]

Question 1

Quelle propriété CSS est utilisée pour créer un conteneur flex ?

- display
- position
- flex-direction

Question 2

Quel est l'axe principal par défaut dans un conteneur flex ?

- Horizontal
- Vertical
- Diagonal

Question 3

Quelle propriété permet de contrôler l'alignement des éléments flex le long de l'axe principal ?

- justify-content
- align-items
- flex-wrap

Question 4

Quelle propriété contrôle si les éléments flex doivent être enveloppés sur plusieurs lignes ou colonnes en fonction de l'espace disponible ?

- flex-direction
- flex-wrap
- align-items

Question 5

Comment sont disposés les éléments flex par défaut lorsque la valeur de « *flex-direction* » est « *column* » ?

- De gauche à droite
- De droite à gauche
- De haut en bas

IV. L'essentiel

Le CSS (Cascading Style Sheets) est un langage essentiel pour transformer les pages web et leur donner un aspect visuel attrayant et interactif. Il permet de styliser les éléments HTML en spécifiant les couleurs, les polices, les tailles, les espacements, etc. En utilisant le CSS de manière appropriée, vous pouvez créer des mises en page esthétiques et réactives pour vos sites web.

L'une des fonctionnalités clés du CSS est le Flexbox, qui permet de créer des mises en page flexibles et réactives. Le Flexbox vous offre un contrôle complet sur la disposition des éléments dans un conteneur, que ce soit horizontalement ou verticalement.

Tous droits réservés à STUDI - Reproduction interdite

Il est important de suivre les bonnes pratiques en matière de CSS pour maintenir un code propre et éviter les problèmes de compatibilité entre les navigateurs. L'utilisation de noms de classes et d'IDs qui ont du sens, l'organisation du code en modules réutilisables et l'utilisation de la cascade de manière judicieuse sont quelques-unes des pratiques recommandées pour garantir une feuille de style CSS bien structurée.

V. Auto-évaluation

A. Exercice

Vous êtes un développeur web travaillant pour une agence de création de contenu numérique qui conçoit des sites web pour divers clients. Vous avez reçu un fichier CSS d'un nouveau collègue qui n'est pas familier avec les bonnes pratiques CSS de votre agence.

Question 1

[solution n°3 p.19]

Pouvez-vous améliorer l'organisation du code en ajoutant des commentaires pour décrire les sections, en regroupant les règles CSS similaires dans des sections thématiques et en suivant les conventions de nommage de votre agence ?

```
body {
font-family: Arial, sans-serif;
background-color: #f0f0f0;
}

h1, h2, h3, h4, h5, h6 {
font-weight: bold;
}

header {
background-color: #333;
color: white;
display: flex;
justify-content: space-between;
align-items: center;
padding: 20px;
}

.logo {
font-size: 24px;
font-weight: bold;
display: flex;
width: 500px;
justify-content: space-between;
}

nav {
display: flex;
justify-content: space-between;
width: 500px;
}

nav a {
color: white;
text-decoration: none;
}

nav a:hover {
color: #f0f0f0;
}
```

Tous droits réservés à STUDI - Reproduction interdite

Question 2

[solution n°4 p.20]

Dans cette même société, on vous demande de concevoir une page d'accueil qui présente 3 produits côte à côte, avec une description et un bouton « *Acheter* » sous chaque produit. Pouvez-vous créer une structure HTML et CSS en utilisant Flexbox pour organiser les produits, les descriptions et les boutons « *Acheter* » ?

```
<!DOCTYPE html>
<html lang="fr">
<head>
 <meta charset="UTF-8">
 <meta name="viewport" content="width=device-width, initial-scale=1.0">
 <link rel="stylesheet" href="styles.css">
 <title>Exemple Flexbox</title>
</head>
  <body>
     <div class="product-container">
     <div class="product">
         <img src="product1.jpg" alt="Produit 1">
        <h3>Description du produit 1</h3>
         <button>Acheter</button>
     </div>
       <div class="product">
         <img src="product2.jpg" alt="Produit 2">
         <h3>Description du produit 2</h3>
         <button>Acheter</button>
     </div>
     <div class="product">
         <img src="product3.jpg" alt="Produit 3">
         <h3>Description du produit 3</h3>
         <button>Acheter</button>
     </div>
      </div>
  </body>
</html>
```

B. Test

Exercice 1 : Quiz

[solution n°5 p.21]

Question 1

Comment débute et se termine un commentaire CSS ?

- `<!-- Commentaire -->`
- `// Commentaire`
- `/* Commentaire */`

Question 2

Qu'est-ce que la spécificité en CSS ?

- La priorité des styles appliqués aux éléments HTML
- La distance entre les éléments HTML
- L'espacement entre les lignes de texte

Question 3

Tous droits réservés à STUDI - Reproduction interdite

Quelle propriété permet de contrôler l'alignement des éléments flex le long de l'axe transversal ?

- justify-content
- align-items
- flex-wrap

Question 4

Pourquoi est-il important de choisir une convention de nommage cohérente pour les classes et les IDs ?

- Pour faciliter la compréhension et la maintenance du code
- Pour réduire la taille du fichier CSS
- Pour améliorer les performances du navigateur

Question 5

Quelle est la principale différence entre les sélecteurs d'IDs et les sélecteurs de classes ?

- Les IDs ont une spécificité plus élevée et ne peuvent être utilisés qu'une seule fois par page
- Les classes ont une spécificité plus élevée et ne peuvent être utilisées qu'une seule fois par page
- Les IDs et les classes ont la même spécificité et peuvent être utilisés autant de fois que nécessaire

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 6 Solution n°1

Question 1

Pourquoi est-il important d'utiliser des commentaires dans votre code CSS ?

- ○ Pour que le navigateur puisse interpréter les styles correctement
- ◉ Pour faciliter la compréhension du code et aider à sa maintenance
- ○ Pour augmenter la vitesse de chargement de la page

Utiliser des commentaires dans votre code CSS permet de décrire les sections du code, de clarifier les intentions du développeur et d'en faciliter la compréhension.

Question 2

Quelle méthodologie de nommage utilise des blocs, des éléments et des modificateurs, séparés par des doubles tirets (--) et des doubles underscores (__) ?

- ○ OOCSS
- ○ SMACSS
- ◉ BEM

BEM (Block, Element, Modifier) est une méthodologie de nommage qui utilise des blocs, des éléments et des modificateurs, séparés par des doubles tirets (--) et des doubles underscores (__).

Question 3

Quelle est l'importance de regrouper les règles similaires et de les séparer par des sections thématiques en CSS ?

- ○ Pour réduire la taille du fichier CSS
- ◉ Pour faciliter la navigation et la maintenance du code
- ○ Pour améliorer les performances du navigateur

Ce regroupement permet de maintenir une organisation lisible du code, ce qui facilite la navigation lors de sa maintenance.

Question 4

Quelle est la spécificité des sélecteurs d'ID par rapport aux classes en CSS ?

- ○ Plus faible
- ○ Égale
- ◉ Plus élevée

Les sélecteurs d'ID ont une spécificité plus élevée que les classes, ce qui signifie qu'ils peuvent prendre le dessus sur les styles définis par les classes lorsqu'il y a un conflit.

Question 5

Pourquoi est-il important d'utiliser des couleurs contrastées en CSS ?

Tous droits réservés à STUDI - Reproduction interdite

- ○ Pour améliorer l'esthétique du site web
- ○ Pour faciliter la navigation sur le site web
- ◉ Pour assurer une bonne lisibilité du texte pour tous les utilisateurs

Utiliser des couleurs contrastées en CSS a pour but d'assurer une bonne lisibilité du texte, particulièrement pour les personnes ayant des déficiences visuelles ou daltoniennes.

Exercice p. 12 **Solution n°2**

Question 1

Quelle propriété CSS est utilisée pour créer un conteneur flex ?

- ◉ display
- ○ position
- ○ flex-direction

La propriété « *display* » est utilisée pour créer un conteneur flex en définissant sa valeur sur « *flex* » ou « *inline-flex* ».

Question 2

Quel est l'axe principal par défaut dans un conteneur flex ?

- ◉ Horizontal
- ○ Vertical
- ○ Diagonal

Par défaut, l'axe principal est horizontal, ce qui signifie que les éléments flex sont disposés horizontalement, de gauche à droite.

Question 3

Quelle propriété permet de contrôler l'alignement des éléments flex le long de l'axe principal ?

- ◉ justify-content
- ○ align-items
- ○ flex-wrap

La propriété « *justify-content* » permet de contrôler l'alignement des éléments flex le long de l'axe principal.

Question 4

Quelle propriété contrôle si les éléments flex doivent être enveloppés sur plusieurs lignes ou colonnes en fonction de l'espace disponible ?

- ○ flex-direction
- ◉ flex-wrap
- ○ align-items

Tous droits réservés à STUDI - Reproduction interdite

La propriété « *flex-wrap* » contrôle si les éléments flex doivent être enveloppés sur plusieurs lignes ou colonnes en fonction de l'espace disponible.

Question 5

Comment sont disposés les éléments flex par défaut lorsque la valeur de « *flex-direction* » est « *column* » ?

- ○ De gauche à droite
- ○ De droite à gauche
- ◉ De haut en bas

Lorsque la valeur de « *flex-direction* » est « *column* », les éléments flex sont disposés verticalement, de haut en bas.

p. 13 **Solution n°3**

```css
/* Reset */
* {
margin: 0;
padding: 0;
box-sizing: border-box;
}

/* General styles */
body {
font-family: Arial, sans-serif;
background-color: #f0f0f0;
}

/* Typography */
h1, h2, h3, h4, h5, h6 {
font-weight: bold;
}

/* Header */
header {
background-color: #333;
color: white;
display: flex;
justify-content: space-between;
align-items: center;
padding: 20px;
}

/* Logo and title */
.logo {
font-size: 24px;
font-weight: bold;
display: flex;
width: 500px;
justify-content: space-between;
}

/* Navigation */
nav {
display: flex;
```

Tous droits réservés à STUDI - Reproduction interdite

```
justify-content: space-between;
width: 500px;
}

nav a {
color: white;
text-decoration: none;
}

/* Navigation links hover state */
nav a:hover {
color: #f0f0f0;
}
```

Dans la solution, nous avons ajouté des commentaires pour décrire les sections et regrouper les règles CSS similaires. Nous avons également ajouté la section « *Reset* » pour réinitialiser les styles par défaut du navigateur.

p. 14

Solution n°4

```
/* Réinitialisation du navigateur */
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: Arial, sans-serif;
}

/* Conteneur des produits */
.product-container {
  display: flex;
  justify-content: space-between;
  flex-wrap: wrap;
}

/* Style des produits individuels */
.product {
  width: calc(33.333% - 20px);
  box-sizing: border-box;
  margin: 10px;
  padding: 20px;
  background-color: #f0f0f0;
  text-align: center;
  border: 1px solid #ccc;
  border-radius: 5px;
}

/* Style des images des produits */
.product img {
  width: 100%;
  height: auto;
}

/* Style du bouton Acheter */
button {
```


Tous droits réservés à STUDI - Reproduction interdite

```
39    background-color: #008CBA;
40    color: white;
41    padding: 15px 32px;
42    text-align: center;
43    font-size: 16px;
44    cursor: pointer;
45    border: none;
46    border-radius: 5px;
47  }
```

La propriété `display: flex` est appliquée au conteneur `.product-container` pour activer le modèle de mise en page Flexbox. Les éléments enfants sont répartis horizontalement avec la propriété `flex-direction: row` par défaut. La propriété `justify-content: space-between` est utilisée pour répartir uniformément l'espace entre les éléments enfants dans le conteneur. Enfin, la propriété `flex-wrap: wrap` est utilisée pour permettre aux éléments de passer à la ligne suivante lorsqu'ils dépassent la largeur du conteneur. Chaque élément `.product` a une largeur définie avec la propriété `width: calc(33.333% - 20px)`, qui permet de diviser uniformément l'espace horizontal entre les 3 éléments.

Exercice p. 14 Solution n°5

Question 1

Comment débute et se termine un commentaire CSS ?

- ○ `<!-- Commentaire -->`
- ○ `// Commentaire`
- ◉ `/* Commentaire */`

Q Les commentaires CSS commencent par `/*` et se terminent par `*/`.

Question 2

Qu'est-ce que la spécificité en CSS ?

- ◉ La priorité des styles appliqués aux éléments HTML
- ○ La distance entre les éléments HTML
- ○ L'espacement entre les lignes de texte

Q La spécificité en CSS détermine la priorité des styles appliqués aux éléments HTML. Les règles de spécificité aident à résoudre les conflits de style lorsque plusieurs règles ciblent le même élément.

Question 3

Quelle propriété permet de contrôler l'alignement des éléments flex le long de l'axe transversal ?

- ○ justify-content
- ◉ align-items
- ○ flex-wrap

Q La propriété « *align-items* » permet de contrôler l'alignement des éléments flex sur un axe vertical.

Tous droits réservés à STUDI - Reproduction interdite

Question 4

Pourquoi est-il important de choisir une convention de nommage cohérente pour les classes et les IDs ?

- [x] Pour faciliter la compréhension et la maintenance du code
- [ ] Pour réduire la taille du fichier CSS
- [ ] Pour améliorer les performances du navigateur

Choisir une convention de nommage cohérente pour les classes et les IDs est important pour maintenir une structure claire et lisible du code, éviter les erreurs, et rendre le code plus facile à comprendre pour les développeurs qui travaillent sur le même projet.

Question 5

Quelle est la principale différence entre les sélecteurs d'IDs et les sélecteurs de classes ?

- [x] Les IDs ont une spécificité plus élevée et ne peuvent être utilisés qu'une seule fois par page
- [ ] Les classes ont une spécificité plus élevée et ne peuvent être utilisées qu'une seule fois par page
- [ ] Les IDs et les classes ont la même spécificité et peuvent être utilisés autant de fois que nécessaire

Les IDs ont une spécificité plus élevée que les classes et sont conçus pour identifier un élément unique sur une page web. Ils ne peuvent être utilisés qu'une seule fois par page.

Tous droits réservés à STUDI - Reproduction interdite